

Team: Schedulix

Abdul Aleem – 1602-23-735-001

Kritika Giridhar – 1602-23-735-018

Rishi N. – 1602-23-735-033

Automotive RTOS Performance & Latency Analyzer

1. Problem

Automotive compute platforms increasingly run multiple real-time workloads on shared multicore processors. CPU utilization alone does not explain why a task was delayed, what caused preemption, how jitter developed, or why a deadline was missed.

- ECU software and validation teams need timing information in terms of tasks, deadlines and scheduling behavior.
- Application logs and low-level RTOS traces often provide separate pieces of evidence.
- The missing step is correlation: connecting what a workload experienced with what the scheduler was doing at the same time.

What conventional monitoring shows	What Schedulix adds
CPU load, isolated timing measurements and raw logs.	Correlated application timing and QNX scheduler evidence linked to ECU workloads.

2. Schedulix in one view

Figure 1. End-to-end Schedulix measurement and analysis flow.

Core idea: Schedulix combines workload timing with QNX scheduling evidence to explain the timing behavior behind the failure.

3. Prototype Architecture

The prototype is implemented as a QNX-native measurement and analysis pipeline. The real-time target collects evidence; analysis and visualization operate on the resulting trace data.

Layer	Prototype implementation
Target	QNX Neutrino 8.0 on Raspberry Pi 4, aarch64, 4 CPU cores.
Workloads	ECU-like periodic QNX/POSIX threads with explicit priority, period and CPU-affinity control.
Instrumentation	Application timestamps for release, ready, execution and completion timing.

Layer	Prototype implementation
Trace collection	Fixed-size 48-byte events, bounded shared-memory MPSC ring buffer and deferred flushing.
Kernel evidence	QNX kernel tracing captured as .kev trace data.
Analysis	Latency, jitter, deadline behavior, CPU utilization, context-switch information and scheduling evidence.
Output	Normalized analysis data consumed by CLI and Qt/QML visualization.

4. Real-Time Workload Model

The current representative workload set is:

Workload	Priority	Period	Role
BRAKE_CTL	20	10 ms	High-priority control workload
ADAS_FUSION	15	20 ms	Perception / fusion workload
DIAG_POLL	10	50 ms	Lower-priority diagnostic workload

5. Trace Correlation



Figure 2. Application timing is correlated with QNX kernel scheduling evidence.

- Application instrumentation shows the timing experienced by each workload.
- QNX kernel tracing provides scheduler and thread-state evidence.
- Correlation places both sources on the same time axis so timing anomalies can be investigated rather than viewed as isolated numbers.

6. Analysis and Root-Cause View

Schedulix converts raw timing events into metrics that are meaningful for real-time software analysis. The analysis focuses on when a workload was released, when it became ready, when it ran, and when it completed.

Timing metrics

Release-to-start latency, release-to-finish response time, execution timing, jitter and deadline lateness.

Scheduling evidence

Thread state changes, context-switch activity and scheduler interactions used to support a root-cause explanation.

7. Qt/QML Visualization

The Qt/QML interface is the presentation layer. It keeps the UI outside the real-time measurement path and presents normalized analysis data as workload metrics, timelines and root-cause information.

[INSERT QT DASHBOARD SCREENSHOT]
Overview / metrics

[INSERT QT TIMELINE SCREENSHOT]
Workload scheduling timeline

Figures 3–4. Prototype Qt/QML views for performance metrics and scheduling timeline.

[INSERT QT ROOT-CAUSE SCREENSHOT HERE]
Root-cause / diagnosis view

Figure 5. Root-cause view showing how timing evidence is presented to the engineer.

8. Prototype Validation

- The prototype was deployed on Raspberry Pi 4 running QNX Neutrino 8.0.
- QNX kernel tracing successfully produced .kev trace data on the target.
- Controlled ECU-like workloads can be executed under defined priorities and periods, with CPU contention used for timing experiments.
- The analysis flow is designed to compare baseline and stressed runs and expose changes in latency, jitter and deadline behavior.

9. What Makes Schedulix Different

Typical monitoring	Schedulix
Reports processor load	Relates processor and scheduling behavior to individual workloads
Shows isolated timing values	Correlates application timing with kernel trace evidence
Reports deadline misses	Investigates the scheduling conditions around the miss
Requires manual trace interpretation	Presents normalized timing and scheduling information through CLI and Qt

Static profiling	Supports repeatable workload and CPU-load experiments
------------------	---

10. Current Scope and Limitations

The prototype is deliberately bounded. The current demonstration uses software CAN simulation and TCP-based CAN-like injection rather than a physical CAN controller. GPIO support includes software/mock capability; physical oscilloscope validation is a future step. These boundaries do not affect the core measurement and correlation concept.

11. Roadmap

Stage	Focus	Outcome
Current prototype	QNX + workloads + tracing + analysis + Qt	Single-target real-time analysis workflow
Next	Physical CAN/UART and GPIO timing validation	Closer hardware-level validation
Scale	Multiple ECU instances and distributed trace correlation	Multi-node timing analysis
Extend	Additional OS/CPU target adapters	Reusable analysis model beyond one target

12. Conclusion

Schedulix addresses a practical problem in real-time automotive software: a missed deadline shows that timing failed, but not why. By combining controlled ECU workloads, application timing and QNX scheduler evidence, the prototype provides a single workflow for measuring, correlating and investigating real-time behavior.